```
        exp = (obs_small[j],
acts[j],rewards[j],act_means[j],qnext[j])
        replay.append(exp)

    return replay
```

- ***1*** Gets observation tensor from team 1, which is a 16 x 13 x 13 x 7 tensor
- ***2*** Gets the list of indices for the agents that are still alive
- ***3*** Subsets the observation tensor to only get the positions of the agents
- ***4*** Gets the list of coordinates for each agent on a team
- ***5*** Decides which actions to take using the DQN for each agent
- ***6*** Adds each individual agent's experience separately to the replay buffer
- ***7*** Loops through each agent

The `team_step` function is the workhouse of the main loop. We use it to collect all the data from the environment and to run the DQN to decide which actions to take. The `add_to_replay` function takes the observation tensor, action tensor, reward tensor, action mean field tensor, and the next state Q value tensor and adds each individual agent experience to the replay buffer separately.

The rest of the code is all within a giant `while` loop, so we will break it into parts, but just remember that it's all part of the same loop. Also remember that all this code is together in Jupyter Notebooks on this book's GitHub page at http://mng.bz/JzKp. It contains all of the code we used to create the visualizations, and more comments. We finally get to the main training loop of the algorithm in listing 9.21.

Listing 9.21. Training loop

```
for i in range(epochs):
    done = False
    while not done:
***1***
        acts_1, act_means1, qvals1, obs_small_1, ids_1 = \
        team_step(team1,params[0],acts_1,layers)
***2***
        env.set_action(team1,
acts_1.detach().numpy().astype(np.int32))    ***3***
```